

Demo: Table Design - Sort Key

Summary

Checked automatic sort keys in Redshift, created custom compound sort keys using `firstname` and `lastname`, and used the `EXPLAIN` command to analyze query execution plans and understand query performance metrics.

Check Existing Sort Key

Target table: `usersauto`

Query Sort Key Information

The screenshot shows the AWS Redshift Query Editor v2 interface. The left sidebar contains navigation options like Editor, Queries, Notebooks, Charts, History, and Scheduled queries. The main editor area displays a SQL query with the following content:

```
1 ---- sort
2 select "column", type, sortkey
3 from pg_table_def where tablename = 'usersauto';
4
5 -- update the sort key
6 create table users_custom_sort sortkey (firstname, lastname) as
7 select * from users;
8
9 select "column", type, sortkey
10 from pg_table_def where tablename = 'users_custom_sort';
11
12 explain SELECT firstname, lastname, total_quantity
13 FROM (SELECT buyerid, sum(qtysold) total_quantity
14       FROM sales
15       GROUP BY buyerid
16       ORDER BY total_quantity desc limit 10) Q, table_design_demo.users_custom_sort U
17 WHERE Q.buyerid = U.userid
18 ORDER BY Q.total_quantity desc;
```

Below the query editor, the results are displayed in a table format. The table has three columns: `column`, `type`, and `sortkey`. The results are as follows:

column	type	sortkey
userid	integer	1
username	character(8)	0
firstname	character varying(30)	0
lastname	character varvind(30)	0

```
SELECT tablename,
       "column",
       sortkey
FROM pg_table_def
WHERE tablename = 'usersauto';
```

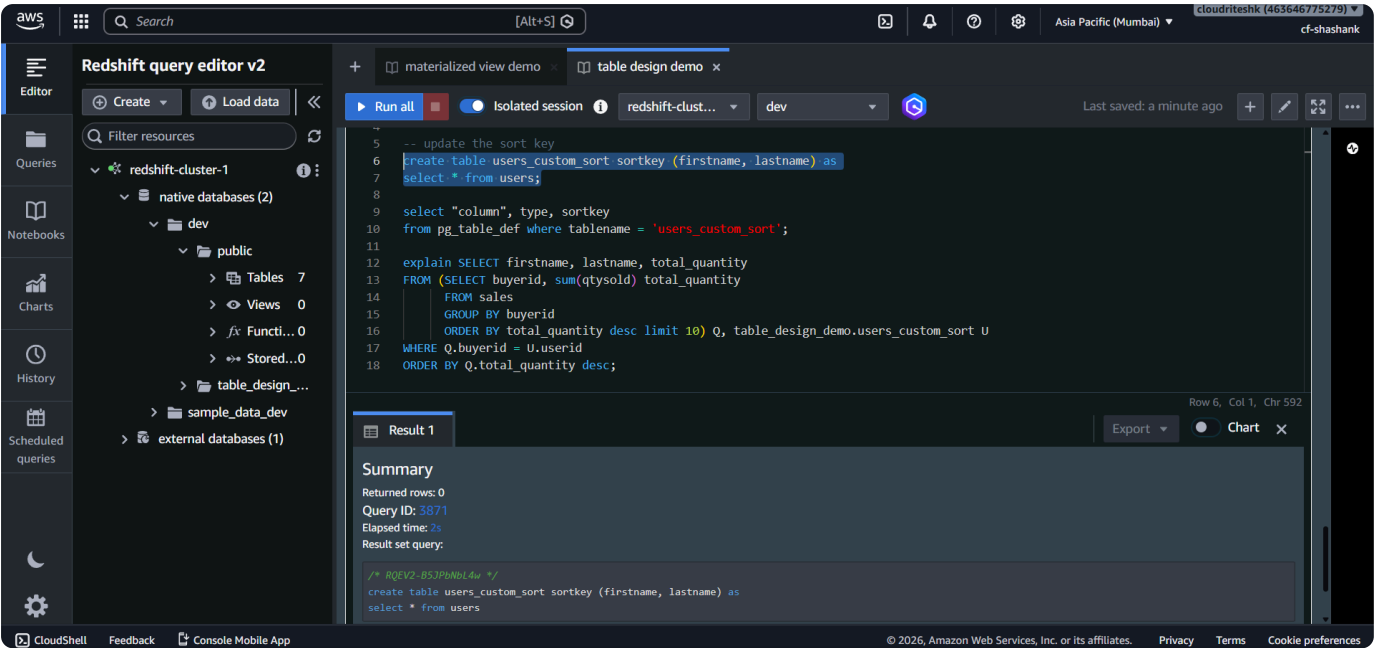
Observation

Redshift automatically assigned: `userid`

as the sort key.

Create Custom Sort Key Table

Create Table with Sort Keys



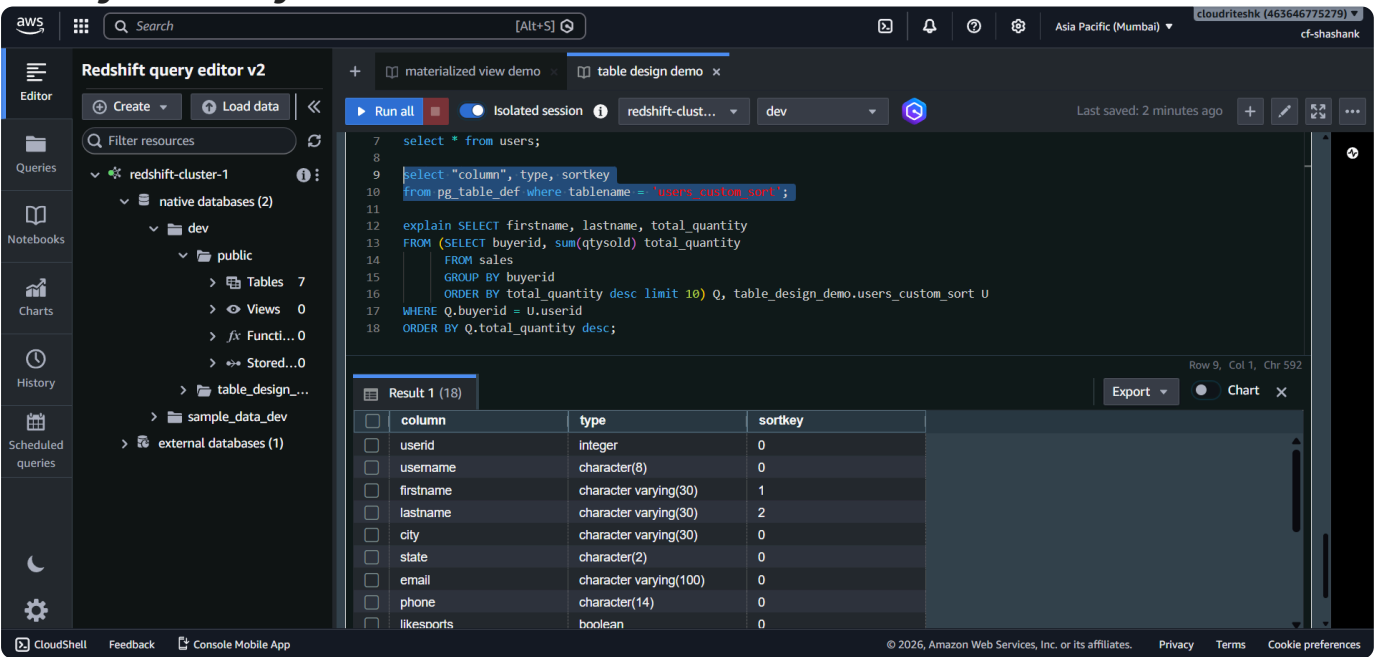
```
CREATE TABLE users_custom_sort
SORTKEY(firstname, lastname)
AS
SELECT * FROM public.users;
```

Important Note

By default: `Compound Sort Key` is created.

You can also use: `INTERLEAVED SORTKEY` if needed.

Verify Sort Keys



```
SELECT tablename,
"column",
```

```
sortkey
FROM pg_table_def
WHERE tablename = 'users_custom_sort';
```

Observation

Sort keys assigned:

- `firstname`
- `lastname`

Query Performance Analysis using EXPLAIN

Purpose

`EXPLAIN` helps analyze query execution plans and performance.

Important

It:

- DOES NOT run the query
- Only analyzes the execution plan

Sample EXPLAIN Query

The screenshot displays the AWS Redshift Query Editor v2 interface. The left sidebar shows the 'Editor' tab with a search bar and a list of resources including 'redshift-cluster-1', 'dev', 'public', 'Tables 7', 'Views 0', 'Functions 0', 'Stored Procedures 0', 'table_design...', 'sample_data_dev', and 'external databases (1)'. The main editor area shows a SQL query with line numbers 11 to 18. The query is an EXPLAIN statement for a SELECT query. Below the query, the 'Result 1 (20)' tab is active, displaying the 'QUERY PLAN'. The plan shows a sequence of operations: XN Merge, Merge Key, XN Network, Send to leader, XN Sort, Sort Key, XN Hash Join DS_BCAST_INNER, Hash Cond, and XN Seq Scan on users_custom_sort u. The bottom status bar indicates 'Query ID: 3871', 'Elapsed time: 18 ms', and 'Total rows: 20'.

```
11 explain SELECT firstname, lastname, total_quantity
12 FROM (SELECT buyerid, sum(qtysold) total_quantity
13 FROM sales
14 GROUP BY buyerid
15 ORDER BY total_quantity desc limit 10) Q, table_design_demo.users_custom_sort U
16 WHERE Q.buyerid = U.userid
17 ORDER BY Q.total_quantity desc;
```

QUERY PLAN

- XN Merge (cost=2000002006456.45..2000002006456.48 rows=11 width=27)
- Merge Key: q.total_quantity
- > XN Network (cost=2000002006456.45..2000002006456.48 rows=11 width=27)
- Send to leader
- > XN Sort (cost=2000002006456.45..2000002006456.48 rows=11 width=27)
- Sort Key: q.total_quantity
- > XN Hash Join DS_BCAST_INNER (cost=1000000005331.38..1000002006456.26 rows=11 width=27)
- Hash Cond: ("outer".userid = "inner".buyerid)
- > XN Seq Scan on users_custom_sort u (cost=0.00..499.90 rows=49990 width=23)

Query ID: 3871 Elapsed time: 18 ms Total rows: 20

```
EXPLAIN
SELECT buyerid,s
       SUM(qtysold)
FROM sales
GROUP BY buyerid
ORDER BY SUM(qtysold) DESC
LIMIT 10;
```

EXPLAIN Output Details

The output includes operations such as:

- Merge
- Network
- Sort

Important Columns in EXPLAIN

Column	Meaning
Cost	Relative execution cost
Rows	Estimated rows returned
Width	Average row size

Cost Values

Cost contains two numbers:

Value	Meaning
First Value	Time to retrieve first row
Second Value	Total time for all rows

Why Sort Keys Matter

Good sort keys help:

- Faster data scanning
- Better filtering
- Improved query performance

You can compare query plans before and after adding sort keys to analyze optimization impact.

Key Learning

Custom sort keys can be created using `CREATE TABLE` or `ALTER TABLE` , and query optimization can be analyzed using the `EXPLAIN` command and execution plan metrics.